Team No: 18	Team Name: HardcoreDesign
Project Title: CapybaraGotchi	
Name: 江善有	ID: 112033254
Name: 林亮宏	ID: 113062339

A. Project description

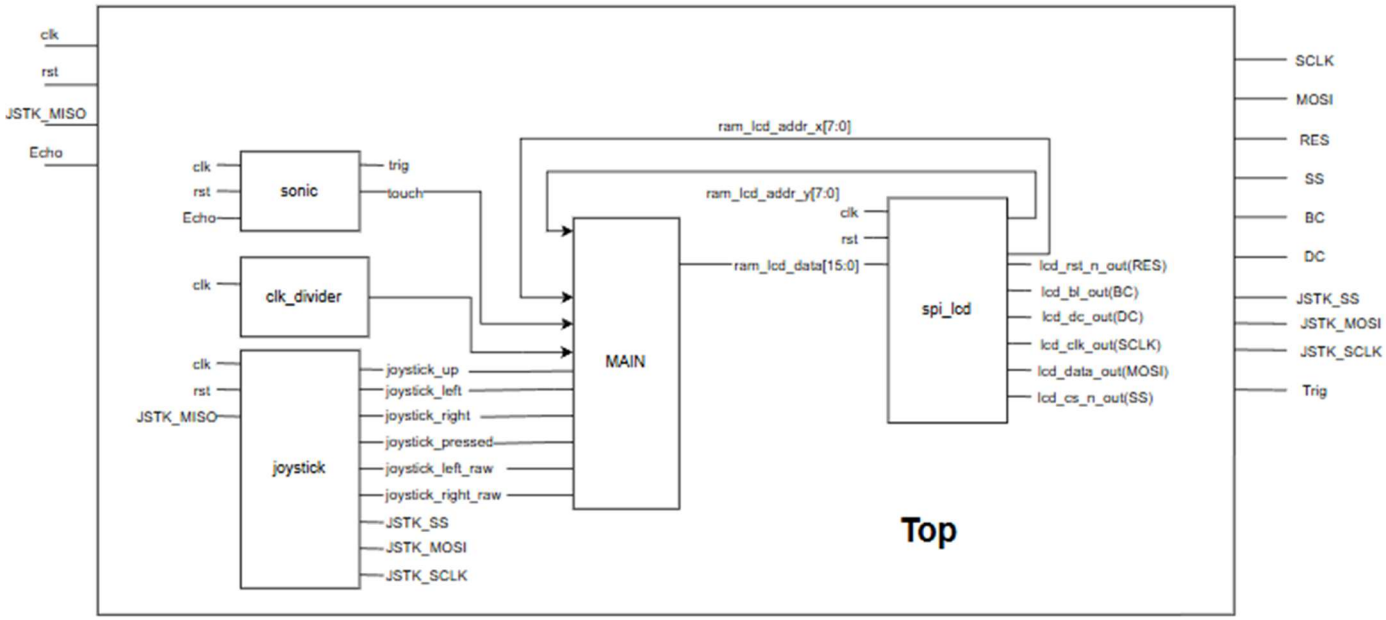
我們這組的主題是電子雞，並將其做成掌上遊戲機的形式。

當初會想要做這個主題是因為看到別人手上拿著的電子雞，當下就覺得其實電子雞就是一個 state machine，很適合使用 verilog 實作出來，而林亮宏又很喜歡使用卡皮巴拉主題的 line 貼圖，所以我們決定客製化的做我們自己喜歡的電子雞。

B. System architect and block diagram

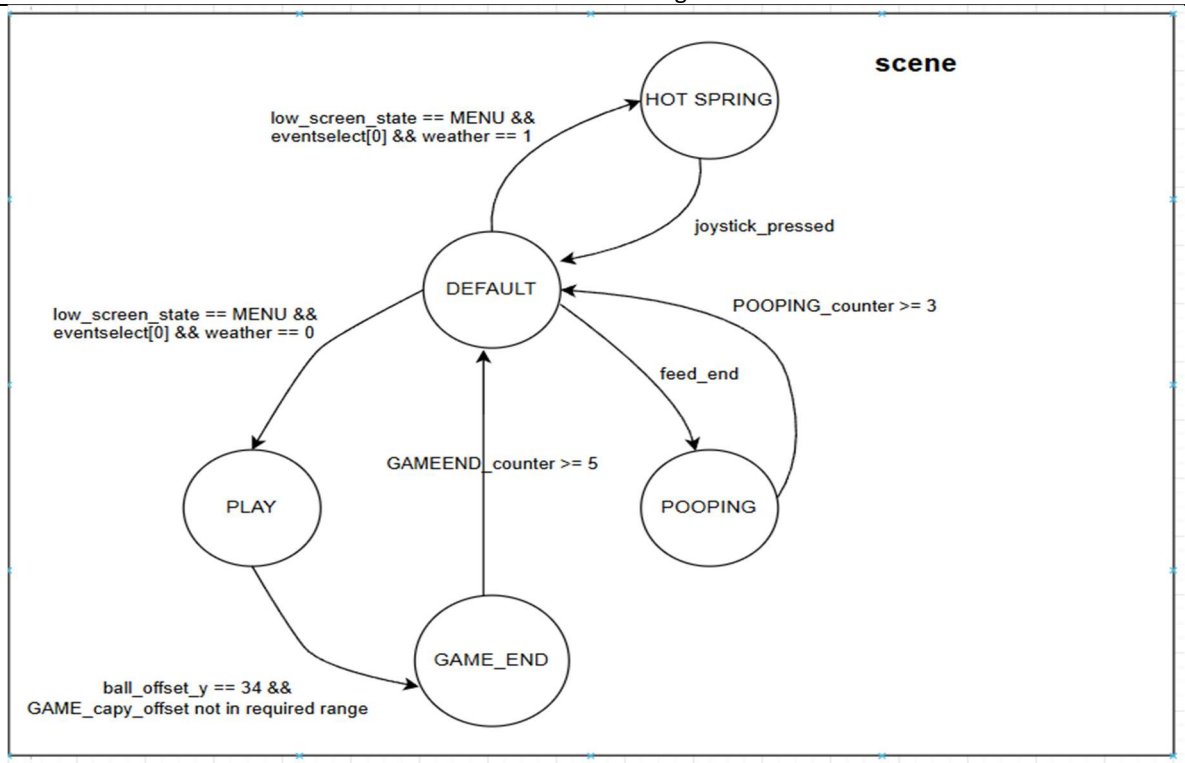
組成的零件為 joystick, LCD 螢幕, ultrasonic sensor 及外殼。Joystick 為玩家主要的操控方式，而 ultrasonic sensor 則是用來實現 touch 功能。

下圖為我們的 top module，joystick module 過修改會傳出 one pulse 過的 up, down, left, right, pressed 等訊號，用來操作互動選單，其中的"raw"訊號表示沒有經過 one pulse，用來實現卡皮巴拉頂球小遊戲內的咖啡杯連續移動功能。LCD module 則是我們用來輸出的模組，可以看到與 vga 類似，都有用來表示座標的訊號和要傳入的顏色訊號，只是這個模組需要 16 bit 的 pixel_data。



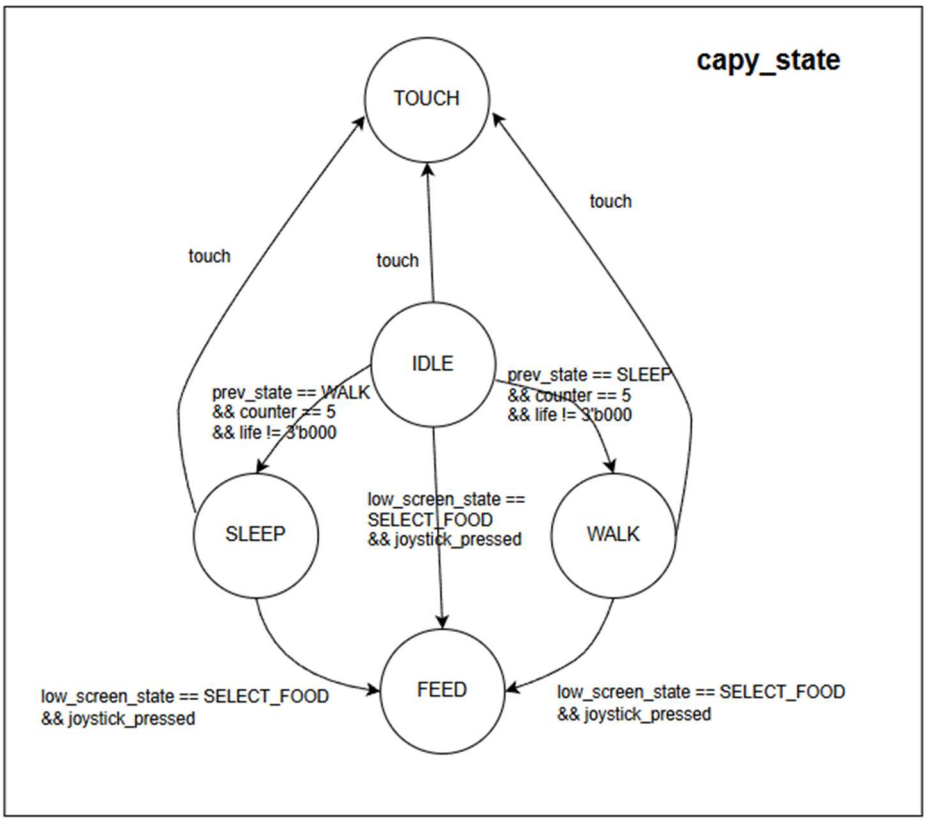
我們的程式部分可以分成三個 FSM，分別是 scene, capy_state, low_screen

1. **scene:** 場景的切換，一般的互動都會在 DEFAULT scene 內進行，泡湯、排泄、玩遊戲、遊戲結束等特別的情況才會進入其他 scene。



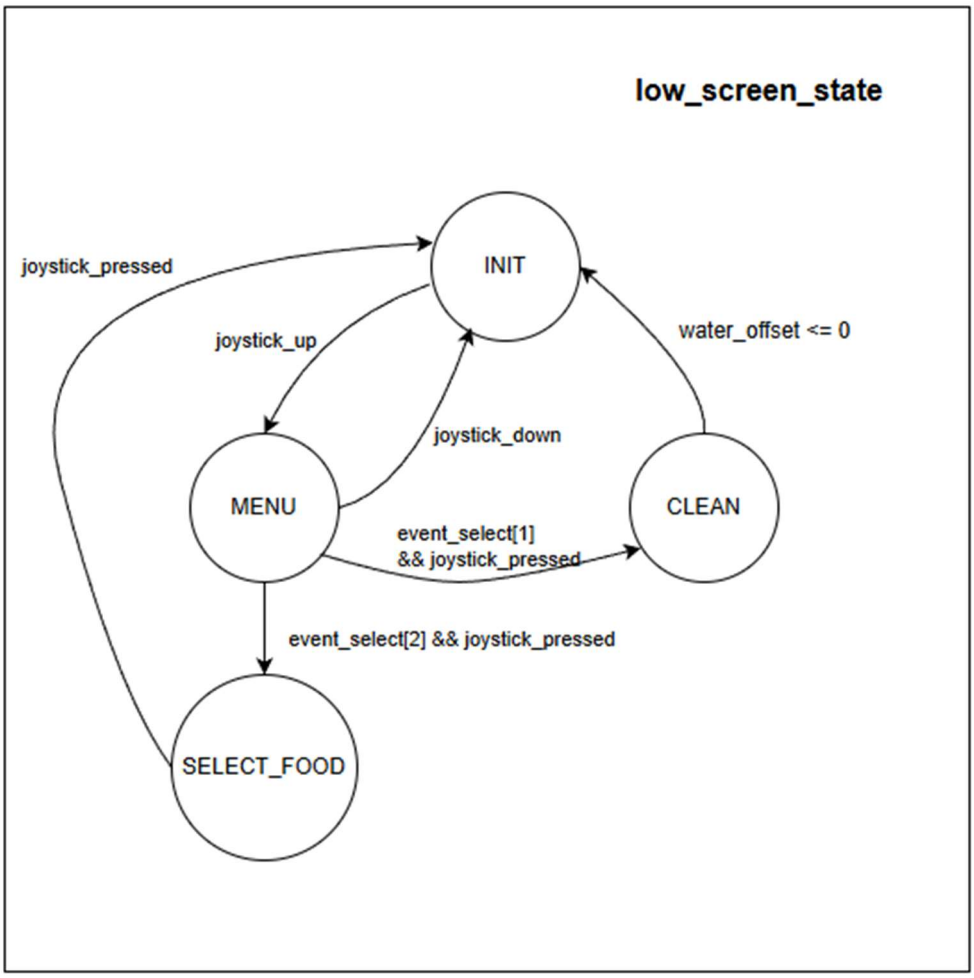
在 **DEFAULT_SCENE** 中，又分為兩個 **FSM** 分別操控中間部分的螢幕及下方區域的螢幕

2. capy_state: 螢幕中間的卡皮巴拉目前所處於的 state



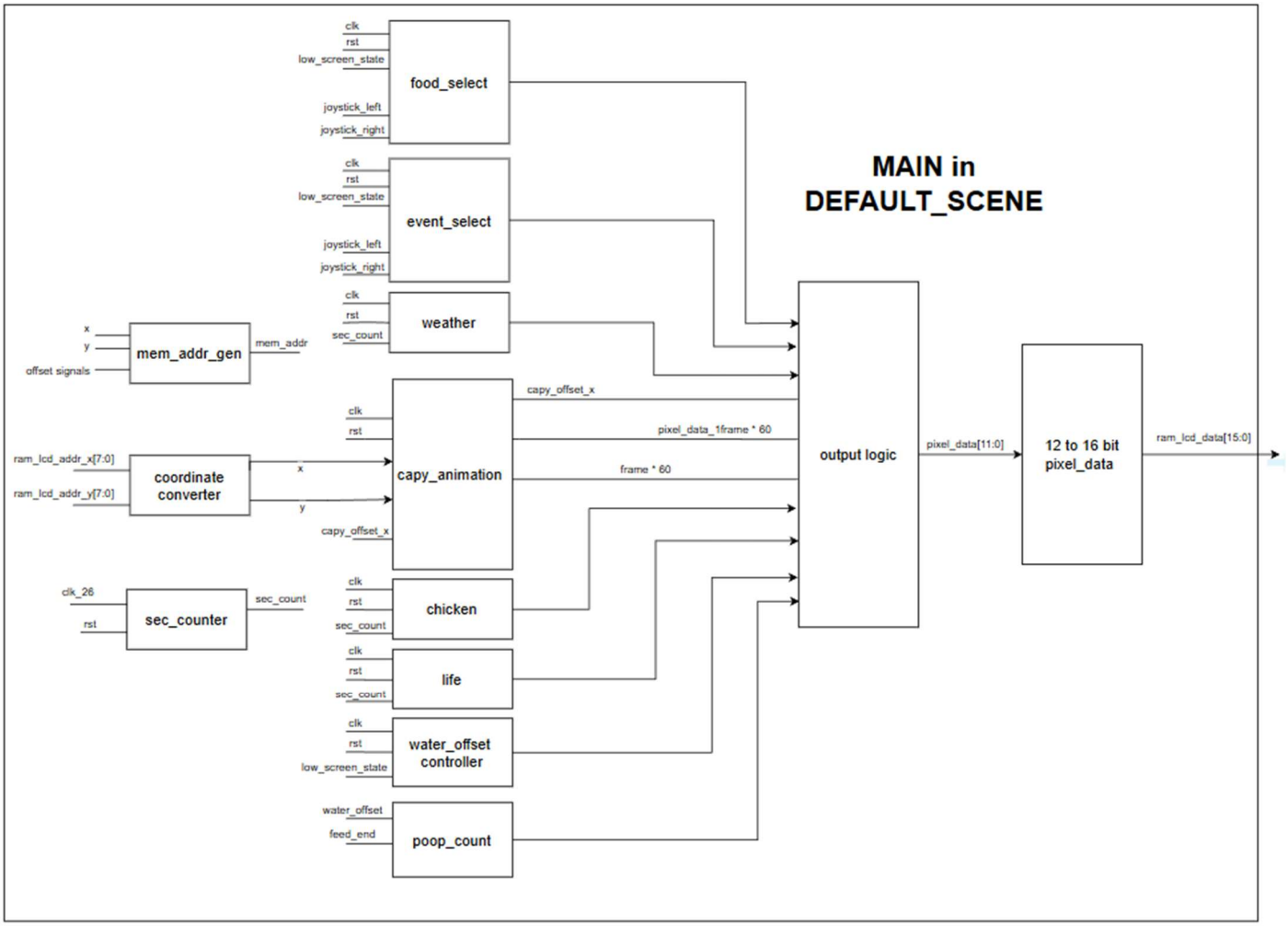
每個 state 中會有不同的動畫，其中 FEED 會另外根據目前的 food_select 來撥放不同的卡皮巴拉吃食物動畫。上圖中的 prev_state 只會紀錄 SLEEP 和 WALK，用來讓兩個 state 交替出現。

3. low_screen_state: 螢幕下方顯示什麼的 state



INIT: 根據 poop_count 顯示排泄物
MENU: 顯示互動選單
SELECT_FOOD: 顯示餵食食物選單
CLEAN: 根據 water_offset 決定要跟 INIT 顯示一樣內容，還是水紋(用來呈現出沖水動畫)。

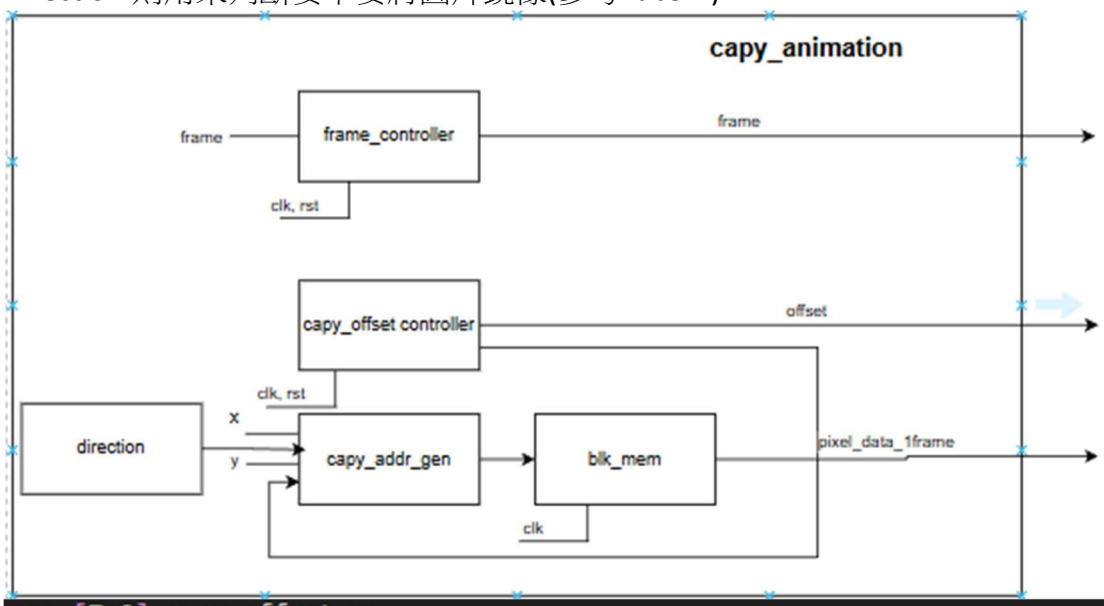
下圖是主要邏輯(MAIN)在 scene 為 DEFAULT 時的 block diagram。我將 capy_animation 這個 block 的訊號畫成 pixel_data_1frame 和 frame 的原因是我們總共使用了 60 個 BRAM，如果要把每個訊號畫上去太耗時而且會非常亂，因此簡化表示。



(i) coordinate converter: 因為我們要將小螢幕放到我們後續設計的機構中，因此我們需要讓座標系變成可以順利放入機構的顯示方向

```
assign rotated_x = 8'd161 - ram_lcd_addr_y;
assign rotated_y = ram_lcd_addr_x;
wire [7:0] x = rotated_x >> 1; // Divide by 2
wire [7:0] y = rotated_y >> 1; // Divide by 2
```

(ii) capy animation: 我們的卡皮巴拉會在螢幕上走動(capy_offset controller)，並且會自動回頭(direction)，動畫則是由一幀一幀的圖片所組成，因此需要 frame controller 和每一個幀的 pixel_data，capy_addr_gen 則是每個圖片會根據位置(capy_offset)的不同數值來計算，而 Direction 則用來判斷要不要將圖片鏡像(參考 lab5-1)




```
reg [7:0] capy_offset_x;
wire [7:0] capy_offset_y = 8'd15;
wire [7:0] capy_img_x = x - capy_offset_x;
wire [7:0] capy_img_y = y - capy_offset_y;
wire [7:0] capy_img_x_final = (Direction == WALK_RIGHT) ? 8'd40 - capy_img_x : capy_img_
wire [7:0] capy_img_y_final = capy_img_y;
wire [20:0] capy_img_addr = capy_img_y_final * 8'd41 + capy_img_x_final;
```

(iii) chicken, life: 飽腹度和生命值，是 shift register 所構成，扣數值左移，加數值右移，並將 MSB 變成 1。當 sec_count 中的 sec_count_DEFAULTSCENE 每到一定數值就會觸發扣數值效果，feed_end 時會增加數值(詳細機制可參考 demo 影片)。

(iv) food_select, event_select: 是 one hot 的 shift register，根據目前的 low_screen_state 來決定會不會被改值

(v) weather: 在 DEFAULT scene 中每十秒切換一次
(vi) water_offset_controller: 初始值為 81，當 low_screen_state == CLEAN 時會減少，用來計算 water 的 memory address 及判斷是否要清空 poop_count 和 state transition
(vii) poop_count: feed_end 時會加 1，當 CLEAN state 結束時(water_offset == 0)，會清 0。

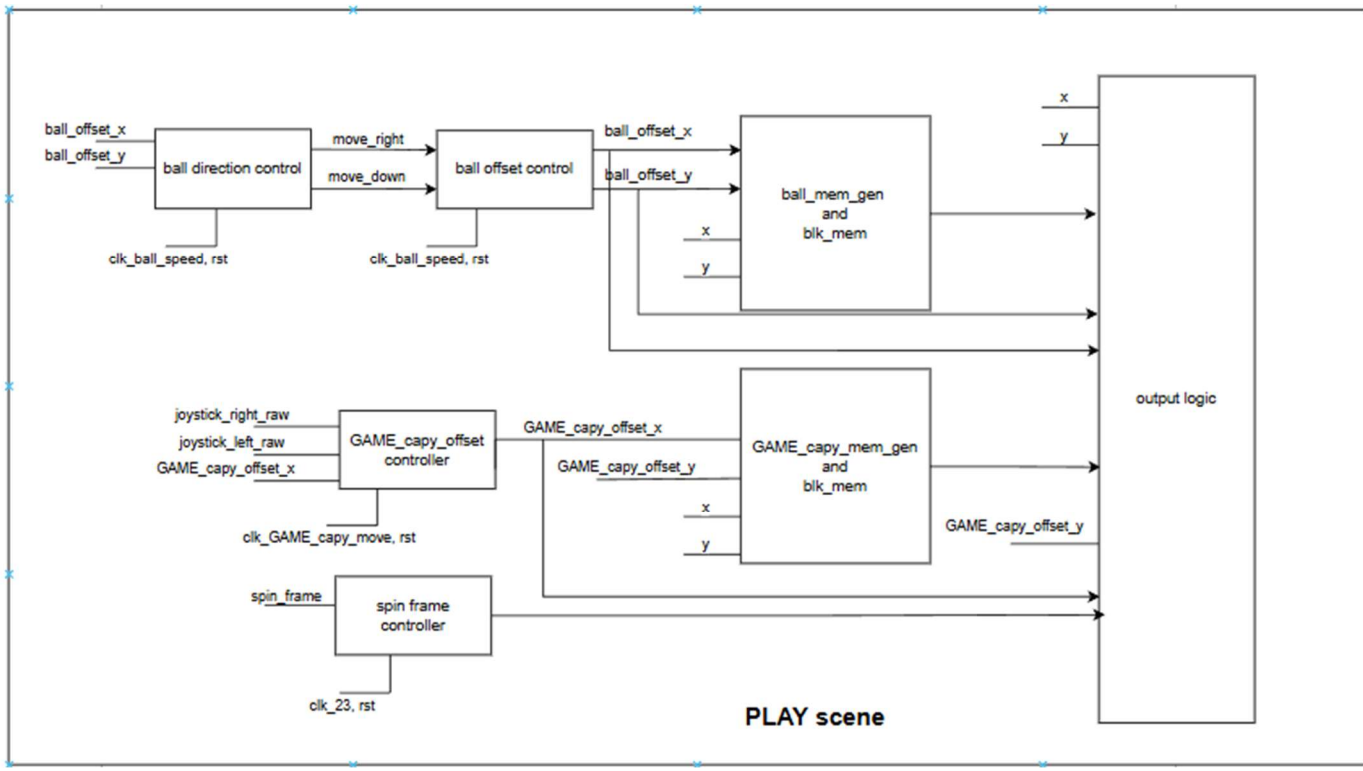
(viii) output_logic, 12 to 16 bit converter: 用上述訊號產出正確的 pixel_data。基本上就是分成一堆 case，assign 不同的 blk_mem 產出的 pixel_data_1frame 給 pixel_data，再轉換成 16bit。

```
SCENE_DEFAULT: begin
    if (y <= 1)
        if (x >= 33 && x < 48) begin
            case(weather)
                0: pixel_data = pixel_data_sun;
                1: pixel_data = pixel_data_snow;
            endcase
        end else begin
            pixel_data = 12'hfff;
        end
    else if (y >= 1 && y < 11) begin
        if (x < 10) begin
            if (life[2])
                pixel_data = pixel_data_heart;
            else
                pixel_data = 12'hfff;
        end else if (x >= 10 && x < 20) begin
            if (life[1])
            pixel_data = pixel_data_heart;
        else
            pixel_data = 12'hfff;
        end else if (x >= 20 && x < 30) begin
            if (life[0])
                pixel_data = pixel_data_heart;
            else
                pixel_data = 12'hfff;
        end else if (x >= 51 && x < 61) begin
            if (chicken[2])
                pixel_data = pixel_data_chicken;
```

```
assign ram_lcd_data ={pixel_data[11:8],pixel_data[11],pixel_data[7:4],pixel_data[7:6],pixel_data[3:0],pixel_data[3]
```

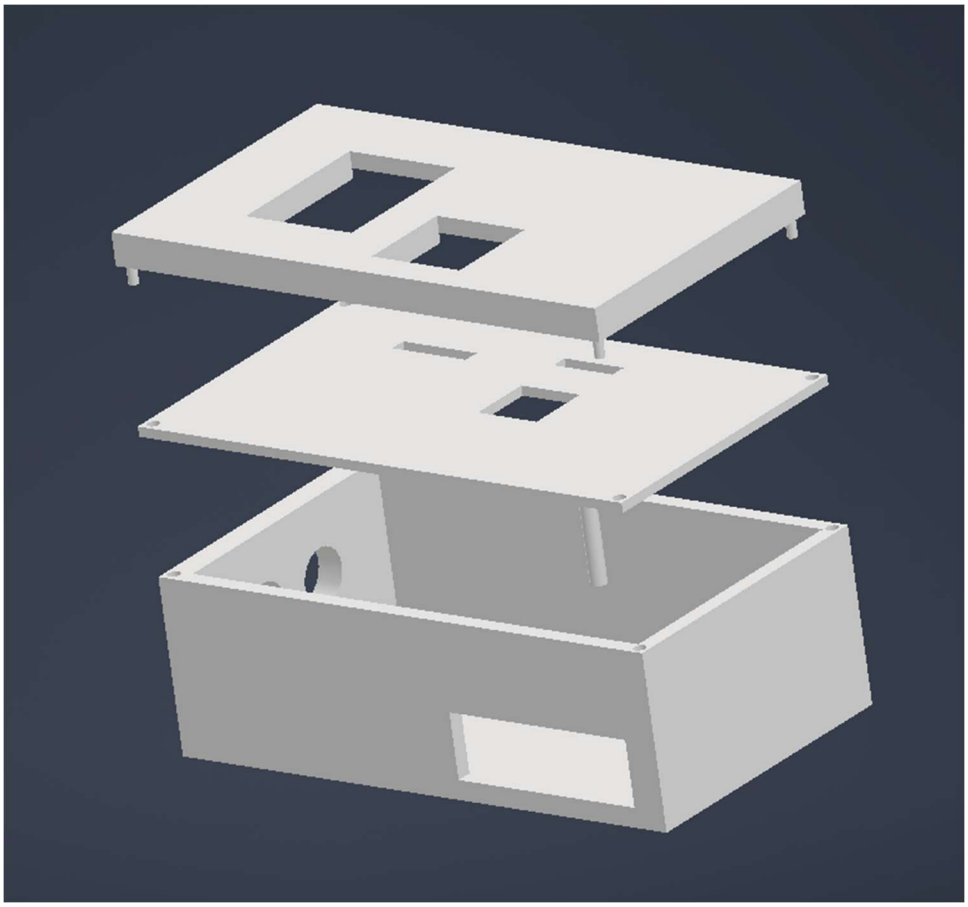
在 GAMEEND、POOPING、HOTSPRING 等 Scene 中所使用的操控方式與 capy animation control 一樣都是使用 frame controller 的方式來做出動畫，因此不再贅述。

PLAY scene



在 PLAY scene 中，卡皮巴拉旋轉杯的方式與 DEFAULT scene 類似，只是透過 joystick 來操作 offset 而不是自動變化，可以看到這邊使用的 joystick 訊號是沒有 one pulse 過的。而球的反彈則是觸摸的邊界就將對應 direction 相反，如果是接觸到卡皮巴拉頭頂的邊界，就觸發判斷卡皮巴拉 offset 的位置是否有頂到球，如果是則反彈，若不是則遊戲結束。

機構設計



分為三層的組合結構(上圖)，四個角的小柱子用於固定 xy 平面不會滑動，因時間有限且需要方便拆卸的考量，我們沒有固定 z 軸的設計，最上層用於固定螢幕和搖桿，中間層用於支撐螢幕和搖桿並讓杜邦線從板子連上來，並加了一個支柱讓板子在內部不會大幅晃動，下層用於容納 FPGA 板和 ultrasonic sensor，挖了洞來放置 ultrasonic sensor 和 program device。

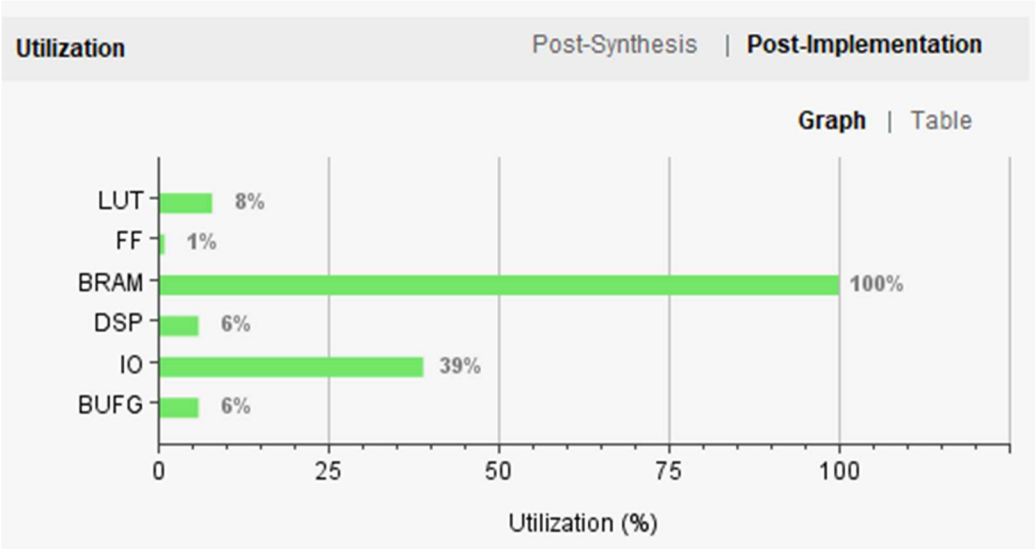
特別感謝 DIT 機器人實驗室借我們列印機使用。

因為列印一次需要 4 個半小時，且我們列印一次要占用兩台機器，所以我們以可以使用為主，預留了一些計算錯誤的容錯率，讓每個固定點都有一個方向的自由度。所幸誤差不大最後列印出來只有四個角的固定孔需要手動鑽大。

C. Implementation progress/ Testing completeness

type	Feature	status	備註
Under planning	IDLE	Complete	
Under planning	SLEEP	Complete	
Under planning	WALK	Complete	
Under planning	SICK	Complete	
Under planning	FEED	Complete	
Under planning	CLEAN	Complete	
Under planning	PLAY(GAME)	Complete	無加速度效果，積分
Additional	weather	Complete	
Additional	Event change(HOTSPRING)	Complete	Based on weather
Additional	Food change(Popsticle)	Complete	Based on weather
Additional	Life and hungry point	Complete	

我們其實是有時間嘗試完成小遊戲的加速度效果和積分的，但我們開始做 weather 後希望可以把整體功能完善，才會比較有掌上型遊戲機的感覺，做完 Additional 的功能後發現 BRAM 被我們用完了，所以就決定先到此為止以外觀的美術為主。



D. Technical challenges and Solutions

因為我們在學期間測試外接設備時就把設計草稿做完了，在實作的過程中沒有遇到太大的困難，基本上就是一些可以觀察出規律的問題，但比較神奇的是我們將 spin_frame 的 clk 用成 clk_23 時，會無法進入遊戲的 state，但改成 clk_25, clk_24 就可以，後續我們又改回 clk_23 又可以了，相當神奇。

初期我們在測試網路上找到的 lcd 螢幕模組時，撇除掉我們看教學文接錯線的部分不看，其他部分都蠻順利的，但我們後續發現該模組只會在 program 完後在初始化的狀態 scan 一次圖片，我們當時是有嚇到因為在這之前都只是測試座標以及顏色的部分，沒想到會發生這種事，好在我們發現只要改掉 MAIN 裡面的 next_state logic，讓 SCAN 可以被重複進入就修好了。下圖為前(左)後(右)對比圖。

```
MAIN:begin
        case(cnt_main)  //MAIN状态
            3'd0: begin
                state <= INIT;
                cnt_main <= cnt_main + 1'b1;
            end
            3'd1: begin
                state <= SCAN;
                cnt_main <= cnt_main + 1'b1;
            end
            3'd2: begin
                // Stay in current state - no loop back to SCAN
                cnt_main <= 3'd2;
                state <= MAIN;
            end
            default: state <= IDLE;
        endcase
    end
```

```
MAIN:begin
        case(cnt_main)  //MAIN状态
            3'd0: begin
            // 第一次進來先做初始化
                state     <= INIT;
                cnt_main  <= 3'd1;   // 初始化做完後，之後都保持在 1
            end
            3'd1: begin
                // 初始化完成後，每次回 MAIN 都再次進 SCAN，重畫整個畫面
                state     <= SCAN;
                // 不要再增加 cnt_main，維持在 1
                cnt_main  <= 3'd1;
            end
            default: begin
                state <= IDLE;
            end
        endcase
    end
```

E. Reflections and Discussions

這次的 project 是我們都認為最好玩、最有成就感的 project，也真的體會到老師說的:「先設計，後續比較容易 debug」的感覺，這次因為我們一開始就把大致的系統架構都設計好了才開始做，所以在實作時就是多加一些細節的訊號和解決一些枝微末節的小問題，並沒有遇到需要大改程式的時候，並且我們保持著新增功能只會增加新訊號或用本來就有的訊號，而不會去改已經有東西在使用的訊號的實作方式，讓整個 project 很好擴充，所以當時林亮宏說泡溫泉的貼圖很可愛時，但不知道這個功能的按鍵要放哪裡，才能想到天氣系統，並且要改的東西也不多，也不會互相影響，一直做到 BRAM 用完才停止。

不過這次專案似乎有很多同學遇到 Access violation 的問題(沒有報錯訊息的 synthesize fail)，我自己是在 advanced lab 就遇過了，當時是王定庭助教和黃威儒助教幫我看問題出在哪的，後續使用助教的 2024 版 vivado 一下就合成成功了，再次感謝助教們的幫助。
因此有同學遇到這個問題時我就跟他們說去試試 2024 版 vivado，也都有成功合成，我自己的觀察是似乎他們都有接鍵盤與使用二維陣列(我自己在 advanced lab 當時也是)，但我並沒有仔細的驗證，只是覺得可以提供一下參考說不定以後就不用有人因此坐牢了(是真的很痛苦，現在看到有錯誤訊息反而覺得人生真美好)。

F. Detailed task division

我們在做這個 project 的時候都是一起待在資電館三樓沙發，就如 demo 時所說，我們先將大概要做甚麼討論好，接著江善有負責寫 code，林亮宏產生圖片的 coe 檔，江善有寫完後如果出現問題，則由林亮宏檢查，經討論後大部分問題幾乎都在半小時內解決。因為我們基本上待在一起做完所有事，所以 contribution 50 / 50。外殼部分則是我們一起想需求與嘗試。

Top module(主要架構)程式碼: 江善有

Top module review: 林亮宏

COE files: 林亮宏

Joystick module : 林亮宏

外殼: both

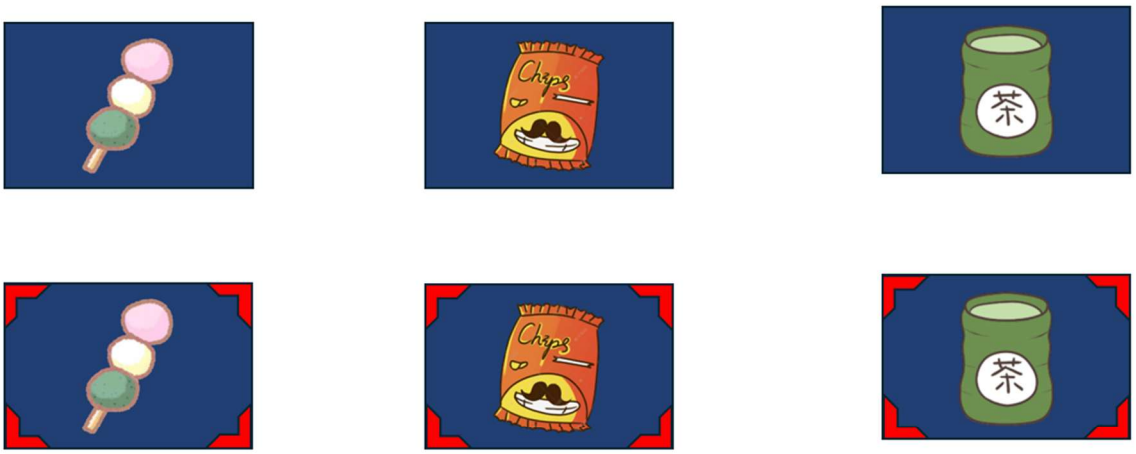
江善有: 50% 林亮宏: 50%

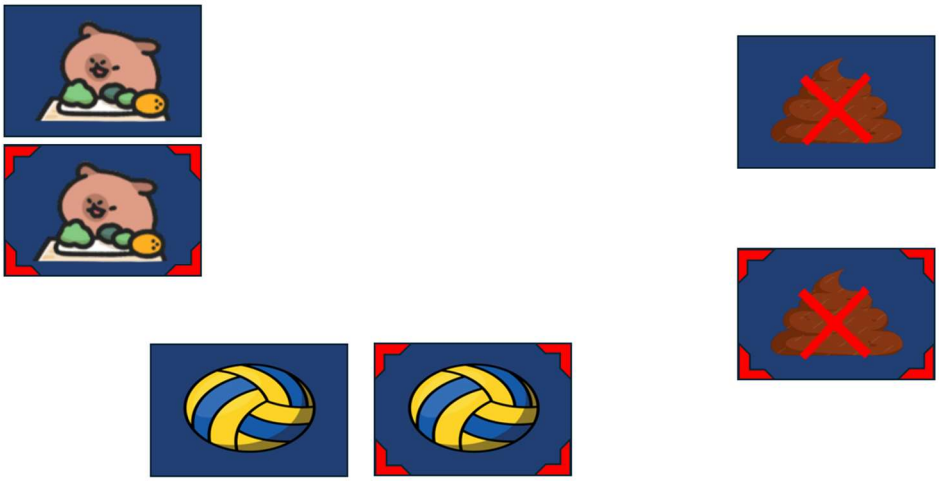
G. Degree of independent work

1. spi_lcd module : found on internet, but can only scan one picture.
   https://chipdebug.com/forum-post/54008.html
   www.stepfpga.com
2. joystick: Original from demo project
   https://digilent.com/reference/pmod/pmodjstk2/start
3. sonic.v: from lab 6 advanced

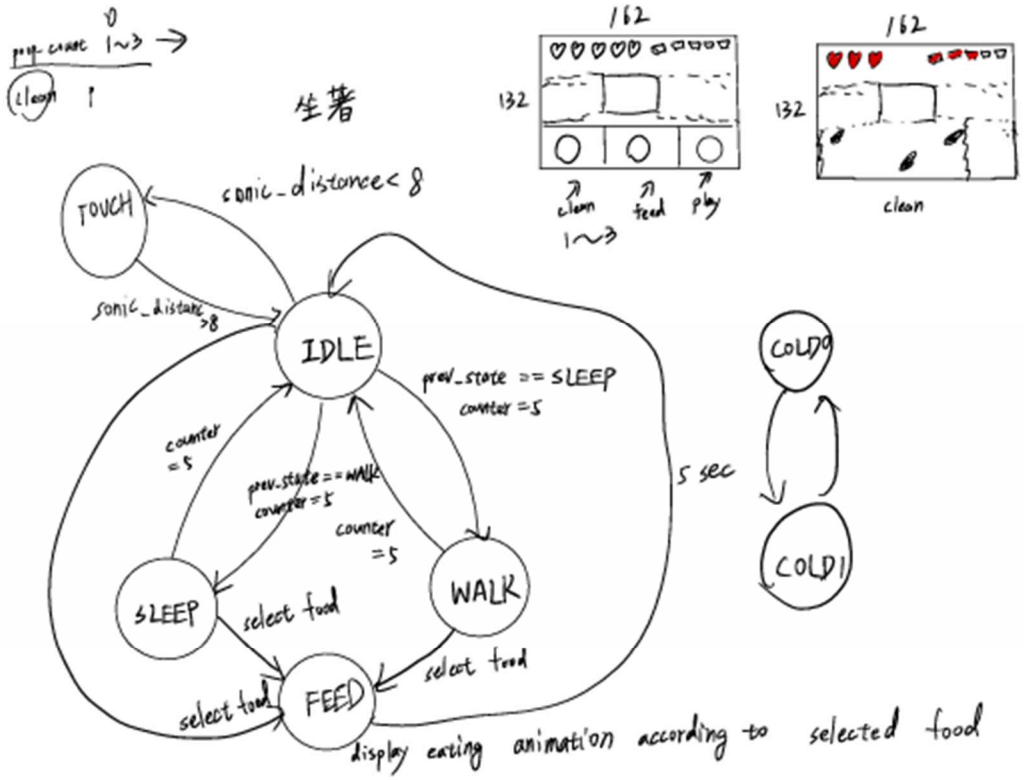
附錄:

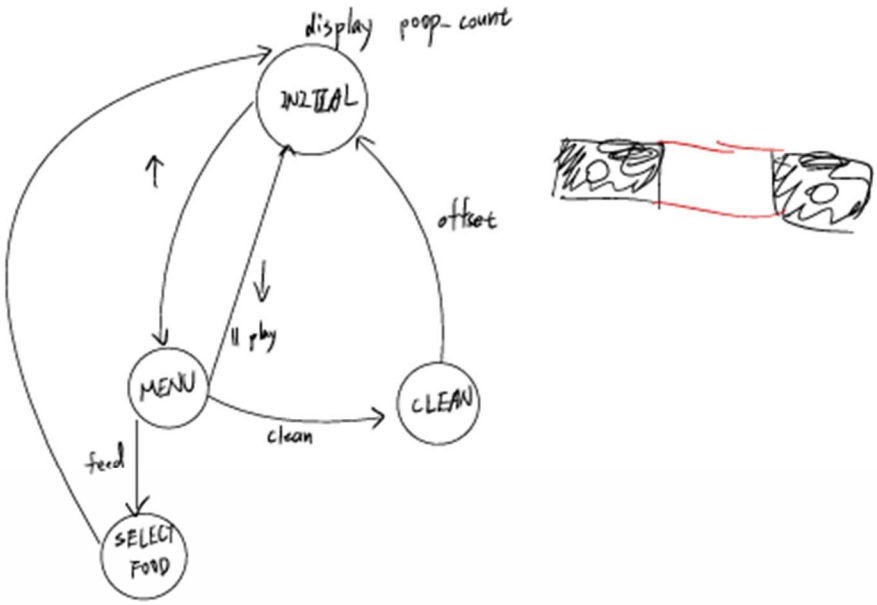
1. menu UI:

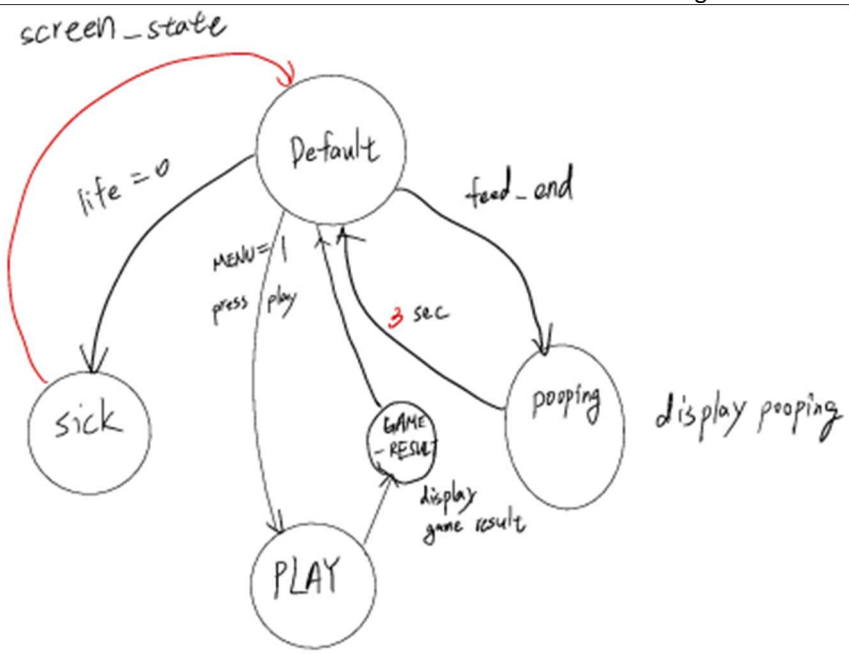


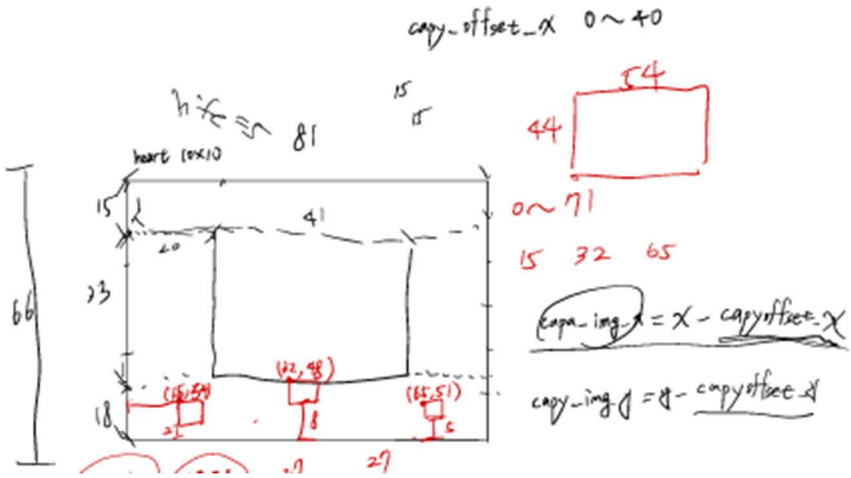
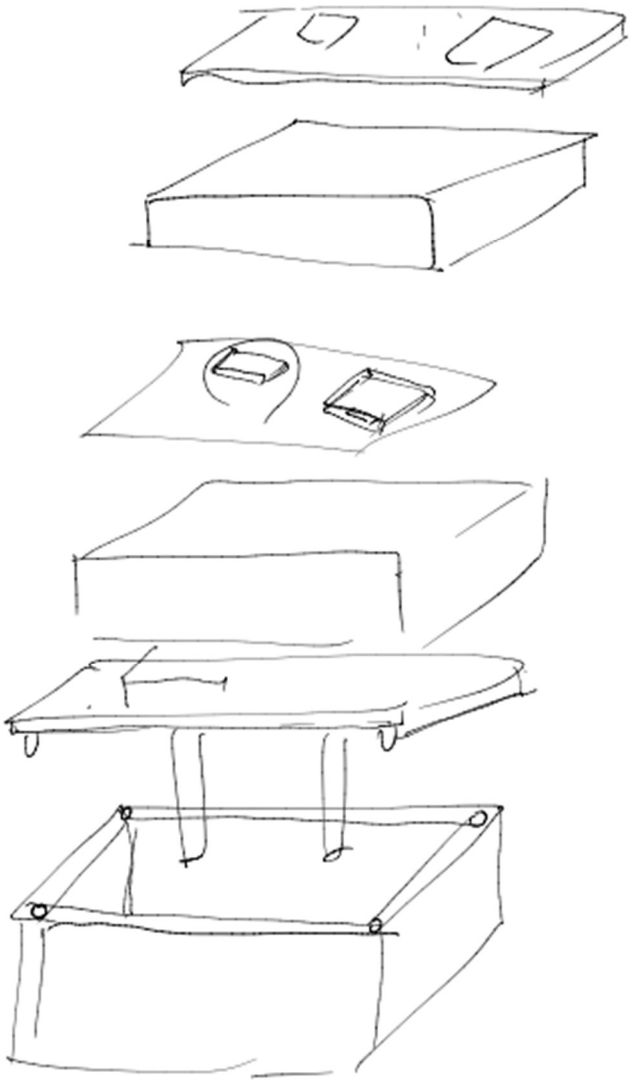


2. 設計手稿:


生著
162
132
clean
feed
play
1~3
clean
TOUCH
sonic_distance < 8
sonic_distance >8
IDLE
prev_state == SLEEP
counter =5
counter =5
prev_state == WALK
counter =5
counter =5
5 sec
SLEEP
select food
WALK
select food
select food
FEED
display eating animation according to selected food
COLD0
COLD1


display poop_count
INITIAL
offset
play
MENU
clean
CLEAN
feed
SELECT FOOD


screen_state
Default
life = 0
feed_end
MENU=1
press play
3 sec
sick
pooping
display pooping
GAME RESULT
display game result
PLAY


copy_offset_x 0~40
54
44
0~71
15 32 65
heart (8x10)
41
20
66
33
18
(32,48)
copy_img_x = x - copyoffset_x
copy_img_y = y - copyoffset_y